

HelixPlay

HelixPlay

Turn any GPU machine into your own cloud-gaming appliance.

Summary

HelixPlay is a self-hostable cloud-gaming platform that turns any GPU-equipped machine into a remote streaming host, delivering console-class gameplay to desktop, mobile, TV, and browser clients. It is built as a Go-centric monorepo of 46 submodules with a triple-stack client and is white-labellable for partners.

Short description

HelixPlay is a self-hostable, open, white-labellable cloud-gaming platform. It turns any GPU-equipped machine into a remote streaming host and delivers console-class gameplay to desktop, mobile, TV, and browser clients over WebRTC/QUIC, with a Go core and a Wails/Flutter/Angular client stack.

Long description

HelixPlay is a cloud-gaming platform built as a Go-centric monorepo composed of 46 Git submodules. It takes any gaming PC you already own and turns it into a streaming host, delivering console-class experience to desktop, mobile, TV, and browser clients — self-hostable, open, and white-labellable for partners. The pitch is blunt: your hardware, your service, your brand, no third-party cloud in the middle.

Its defining design choice is triple-stack client convergence — the hard architectural bet that pays off everywhere else. A Wails desktop app, a Flutter mobile/TV app, and an Angular web client all sit on top of a *single* Go core, compiled to WASM for the browser, so behaviour is written once and shared across every

surface instead of forked three ways. Under that lives the real-time media path: Capture → Encode → Packetize → Transmit → Decode → Render, wired to platform-native capture (DXGI / ScreenCaptureKit / PipeWire) and hardware encoders (NVENC / QSV / AMF / VideoToolbox) so the GPU does the heavy lifting, and transported over WebRTC (Pion v4), QUIC (quic-go), and custom UDP datagrams chosen for latency rather than convenience. The backend core owns sessions, tenants, catalog, and auth; a host agent handles capture, encode, and transport at the edge; and mDNS/rendezvous stitches discovery together so clients find their host without manual wiring.

HelixPlay is architected from the ground up for white-label SaaS — per-tenant theming, catalog filtering, OAuth2, and billing — so a partner can stand up a fully branded service rather than a thin reskin. And it is container-native to the last detail: every service, database, build, test, and scan runs inside containers, which makes the whole platform reproducible to deploy and to verify. Like the rest of the Helix family, it lives under an anti-bluff constitution where a green test is meant to guarantee real, end-user-usable behaviour — not a passing mock.

Why we built it

Commercial cloud gaming is closed, centralized, and rented. HelixPlay was built so anyone with a GPU machine can run their own streaming host — open, self-hosted, and white-labellable — instead of depending on a third-party service.

Why it's a game-changer

It fuses three things the commercial services keep apart: self-hosting on hardware you control, one shared Go core driving three client stacks so features land everywhere at once, and white-label multi-tenancy. The result is that a partner can launch a fully branded cloud-gaming service on their *own* GPUs — owning the experience, the users, and the economics — instead of reselling capacity on someone else's cloud and living inside its limits.

What's innovative

- **Triple-stack client convergence** — Wails, Flutter, and Angular all riding one Go core (WASM in the browser), so desktop, mobile, TV, and web share a single implementation instead of three drifting ones.

- **Self-hostable, white-label SaaS** — per-tenant theming, catalog filtering, OAuth2, and billing built in, so the platform ships as a brandable product, not a demo.
- **Modern low-latency transport** — WebRTC (Pion), QUIC, and custom UDP paired with per-platform hardware-encoder selection (NVENC / QSV / AMF / VideoToolbox), tuned for responsiveness over convenience.
- **46-submodule decoupled architecture** — cleanly separated components with container-native everything: each service, DB, build, test, and scan runs in a container.

Biggest technical challenges & how we solved them

- **Low-latency streaming across heterogeneous hardware.** Every OS and GPU exposes capture and encode differently, and latency is unforgiving. Solved with a platform-aware capture/encode path — DXGI / ScreenCaptureKit / PipeWire feeding NVENC / QSV / AMF / VideoToolbox — carried over WebRTC / QUIC / UDP so each machine uses its own fastest native route to the pixels.
- **One product across desktop, mobile, TV, and web.** Solved with triple-stack clients (Wails, Flutter, Angular) sharing a single Go core compiled to WASM for the browser, so a fix or feature written once shows up on all four surfaces instead of being ported four times.
- **Multi-tenant white-label operation.** Solved by building per-tenant theming, catalog filtering, OAuth2, and billing directly into the core backend, so tenant isolation and branding are platform primitives rather than per-customer forks.

Tech stack

- **Go (1.26.2 root / 1.25+ submodules)** — the shared core backend and host agent; one language that compiles to native binaries *and* to WASM, which is what makes the single-core, multi-client design possible.
- **Wails v2** — the desktop client, binding the Go core to an embedded webview so the desktop app reuses core logic directly instead of reimplementing it.
- **Flutter 3.29+** — the mobile/TV client, calling into the Go core over FFI for a native UI on phones and televisions without a second backend.
- **Angular 17+** — the web client, running the very same Go core compiled to WASM so the browser is a first-class surface, not a stripped-down one.

- **WebRTC / Pion v4, QUIC / quic-go, custom UDP** — three real-time transports so the platform can pick the lowest-latency path available for each network and client.
- **Hardware encoders (NVENC / QSV / AMF / VideoToolbox) and platform capture (DXGI / ScreenCaptureKit / PipeWire)** — the GPU-accelerated capture-and-encode path, selected per platform so encoding never bottlenecks on the CPU.
- **Containers (Docker/Podman)** — every service, database, build, test, and scan runs containerized, making the whole system reproducible to deploy and to verify.
- **mDNS / rendezvous** — zero-config host discovery so clients locate their streaming host automatically on the network.

Status & honesty notes

- **Status: beta.** The README's latency targets (≤ 30 ms LAN / ≤ 50 ms WAN p999), "console-class / PS4-Pro-class" framing, and the test-matrix cell count are the project's own stated design targets, not independently benchmarked, and are presented as such.
- **License: TBD.** No LICENSE was detected via the GitHub API — UNVERIFIED / not declared.

Priority tier: Helix-primary.